

Fragment-Chained HMAC Authentication with Selective Encryption for JPEG Transmission over LoRa Mesh

August Chao

Department of Computer Science and Information Engineering,
National Pingtung University, Pingtung 90003, Taiwan
augchao@gms.npu.edu.tw

Abstract—Transmitting JPEG images over LoRa mesh networks is challenging: the 255-byte payload limit and ETSI 1% duty-cycle constraint impose severe bandwidth and latency budgets while security requirements demand both confidentiality and integrity protection across tens of fragments. Existing approaches apply full AES-128-CBC encryption and independent per-packet HMAC, which (i) incurs unnecessary CPU cost when only visually sensitive bytes require confidentiality, and (ii) cannot detect fragment reordering or cross-session replay because each packet is verified independently.

This paper makes four contributions. First, we design a *stride-based selective encryption* scheme that XORs the first 16 bytes of every 64-byte window with an HMAC-SHA256 keystream, encrypting 24.2% of image bytes and achieving format-level visual obfuscation (JPEG decode fails under both naive and format-aware attacker models) while reducing encryption CPU cost by 76% compared to full encryption (2,807 μ s vs. $\approx$ 11,600 μ s estimated for the same image size). Second, we introduce *Fragment-Chained HMAC* (FC-HMAC), an 8-byte truncated chain where each fragment's tag covers the previous tag, binding the entire transmission sequence into a single cryptographic commitment. Third, we formally verify FC-HMAC using Tamarin Prover: all four lemmas (fragment authenticity, chain-gap detectability, no-replay, executability) are machine-proved in 1.53 seconds under a Dolev-Yao adversary. Fourth, we validate the scheme on real LoRa hardware (EoRa PI: ESP32-S3 + SX1262): 72/72 fragments verified in hardware over a 120-second run, with 3.3% HMAC overhead per packet and 3.76 ms total crypto CPU per 2,046-byte image at identical airtime (14.9 s) across all three encryption modes.

Index Terms—LoRa, LPWAN, image transmission, selective encryption, fragment chaining, HMAC, formal verification, Tamarin Prover, ESP32-S3, IoT security

I. INTRODUCTION

Low-Power Wide-Area Networks (LPWANs), and LoRa in particular, have emerged as a dominant radio layer for battery-operated IoT sensors that need to communicate over distances of several kilometres on harvested or coin-cell power [1]. The physical layer chirp-spread-spectrum (CSS) modulation trades data rate for link budget, achieving -148 dBm sensitivity at SF12 at the cost of throughputs as low as 250 bps. Within the 255-byte hardware payload ceiling of the Semtech SX1262 [2] and the ETSI 1% duty-cycle rule, a single LoRa node in the 915/868 MHz ISM band may inject only ≈ 36 packets per hour at spreading factor 9 (SF9).

A growing class of edge applications requires *image* transmission over these links: wildlife camera traps that report

motion events to a gateway, border-surveillance fixed nodes that capture a thumbnail on alarm, disaster-area drone relays that upload reconnaissance snapshots, and smart-agriculture sensors that record crop canopy images for periodic analysis. For these use cases, a compact JPEG image (200×150 pixels, ≈ 2 KB) is a practical and sufficient representation. Transmitting this image over LoRa at SF9/BW125/CR4/7 requires approximately 9 fragments and roughly 14.9 seconds of airtime — within budget for one capture every few minutes, but leaving no room for heavy cryptographic overhead on the critical path.

The Security Gap in Prior Work

Existing security schemes for LoRa image transmission [3] apply *full* AES-128-CBC encryption to the complete image payload, pairing it with *independent* per-packet HMAC-SHA256. While this approach has been validated at the hardware level and demonstrates negligible energy overhead, it exhibits two structural weaknesses that limit its applicability to more adversarial deployment environments:

CPU inefficiency. Full AES-128-CBC encryption of a 2,046-byte image on an ESP32-S3 at 240 MHz requires approximately 11,600 μ s of CPU time. On a duty-cycle-constrained node that must also handle radio interrupts, sensor sampling, and routing, this represents a significant fraction of the available compute budget. When the goal is merely visual obfuscation — preventing an interceptor from reconstructing a recognisable image — encrypting every byte is wasteful, because a small fraction of structurally critical bytes (JPEG format headers, DCT DC coefficients) suffices to render the image unrecoverable.

Order-agnostic integrity. Independent per-packet HMAC verifies each fragment in isolation, providing *existential unforgeability* for each individual packet. However, an attacker with access to a previous valid capture session may:

- 1) *Replay* an entire valid session from a previous capture without triggering integrity failures at the receiver — because no MAC field commits to the *session context*.
- 2) *Reorder* fragments within a session and deliver them out of sequence; each fragment passes its individual HMAC check even though the assembly order has been corrupted.

- 3) *Insert* a forged fragment from a different session at any position; the receiver cannot distinguish this from a legitimately received fragment based on per-packet HMAC alone.

These vulnerabilities are not theoretical: in a LoRa mesh deployment, a Dolev-Yao adversary who controls the radio channel can store-and-forward any received fragment to any subsequent session [4].

Contributions

This paper addresses both weaknesses through the design, formal analysis, and hardware validation of the *Fragment-Chained HMAC* (FC-HMAC) scheme combined with stride-based selective encryption. The contributions are:

C1 — Stride-based selective encryption. We design a lightweight structure-approximating encryption scheme that XORs the first 16 bytes of every 64-byte window with an HMAC-SHA256 keystream, encrypting 24.2% of image bytes. The scheme achieves *format-level visual obfuscation*: the stride begins at byte 0 (the JPEG SOI marker, 0xFF 0xD8) and corrupts Huffman table regions, breaking JPEG structure entirely under both a naive attacker and a format-aware attacker who zeroes the encrypted positions. We explicitly acknowledge that this is *not* true frequency-domain selective encryption (which requires a Huffman-stream parser to locate DCT DC coefficient bytes at variable positions in the bit-stream); the contribution is a computationally cheap proxy that achieves the same visual disruption at 24% of full-encryption CPU cost.

C2 — Fragment-Chained HMAC (FC-HMAC). We introduce an 8-byte truncated chained MAC where $tag_i = \text{HMAC-SHA256}(K, frag_id_i \parallel flags_i \parallel payload_i \parallel tag_{i-1})[8]$. The chain makes every tag a commitment to the entire prior transmission sequence, enabling detection of any reordering, insertion, or cross-session replay at 3.3% per-packet overhead (8 B / 241 B).

C3 — Formal verification. We model FC-HMAC in Tamarin Prover [4] and machine-prove four security properties: fragment authenticity (F1), chain-gap detectability (F2), no-replay (F3), and protocol executability (F4). All four lemmas are proved in 1.53 seconds under a Dolev-Yao adversary.

C4 — Hardware validation. We implement the full protocol on EoRa PI hardware (ESP32-S3 + SX1262, EoRa PI development board), transmitting a 2,046-byte synthetic JPEG at 922.0 MHz over a real LoRa channel. In a 120-second run with 8 complete images, 72/72 fragments pass chain verification (chain_ok = 100%, PDR = 100%). Total crypto CPU is 3,757 μ s per image, representing 3.1% of a 120-ms processing loop, with 3.3% per-packet HMAC overhead.

Scope and Reproducibility

The experiments in this paper use a 2,046-byte synthetic JPEG embedded in firmware flash. The choice of a synthetic image is deliberate: it enables reproducible measurements independent of camera hardware variability and lighting conditions. The byte array is generated offline from a known-content 200×150 pixel test pattern using the libjpeg library

at quality factor 75. All firmware source code, Tamarin model files, PSNR evaluation scripts, and raw E19 measurement data will be made publicly available via the project repository upon paper acceptance.

Paper Organisation

Section II reviews LoRa constraints, JPEG structure, and the FC-HMAC concept. Section III presents the detailed system design including threat model, the selective encryption algorithm, FC-HMAC construction, and packet format. Section IV gives the Tamarin formal analysis and E19-PSNR visual security evaluation. Section V reports hardware platform, E19 experiment results, fragment timing, and efficiency analysis. Section VI compares FC-HMAC with related work, including a seven-dimension table against Ryzek & Sobieraj [3]. Section VII discusses limitations and future directions. Section VIII concludes.

II. BACKGROUND

A. LoRa Physical Layer and Protocol Constraints

LoRa is a chirp-spread-spectrum modulation scheme defined by Semtech and standardised within the ETSI EN 300 220 framework [1]. Three parameters dominate the link budget and throughput tradeoff:

Spreading Factor (SF). SF ranges from 6 to 12. Each increment doubles symbol duration ($T_s = 2^{SF}/BW$), increasing receiver sensitivity by ≈ 2.5 dB at the cost of halving the data rate. At SF9/BW125, symbol duration is $T_s = 4.096$ ms, giving a raw bit rate of $\approx 1,758$ bps.

Payload ceiling. The SX1262 radio buffer is 255 bytes [2]. After subtracting the FC-HMAC header (7 B) and authentication tag (8 B), the maximum application payload per packet is 240 B.

Duty-cycle regulation. ETSI EN 300 220 mandates a 1% maximum on-air duty cycle in Sub-GHz ISM bands. At SF9/BW125/CR4/7 with a 248-byte payload, measured airtime per fragment is 1,692 ms. The 1% rule therefore limits a single node to approximately 6 transmissions per 10-minute window — sufficient for one 9-fragment image every ~ 4 minutes.

These constraints collectively motivate (i) compact per-packet authentication overhead to preserve payload capacity and (ii) CPU-efficient encryption so that the crypto latency does not dominate the inter-packet gap.

B. LoRaWAN and LoRa Mesh Security Context

LoRaWAN, the most widely deployed LoRa network architecture, provides session-level AES-128 encryption and per-message MICs at the network layer [1]. However, LoRaWAN is a star-topology protocol requiring a centralised network server and dedicated gateway infrastructure. Multi-hop LoRa mesh protocols, such as LoRaMesher [5], operate in infrastructure-free environments where nodes relay packets for one another using distance-vector routing. In these deployments, the LoRaWAN session key infrastructure (AppSKey, NwkSKey) is unavailable; nodes rely on pre-shared application keys for security, and the security of the fragmentation

protocol — which LoRaWAN does not define — is left entirely to the application layer.

The threat surface in a LoRa mesh is broader than in LoRaWAN because relay nodes are themselves potential adversaries: a compromised relay can silently drop, modify, or replay fragments it forwards, with no gateway to detect the anomaly. FC-HMAC is designed with this threat model explicitly in mind.

Previous work [3] demonstrates that application-layer AES-CBC plus per-packet HMAC is feasible on ESP32-class hardware with <1% energy overhead for 20 kB images. Our work examines whether the per-packet HMAC architecture can be strengthened to detect ordering and replay attacks, and whether selective encryption can reduce the CPU cost at the expense of encrypting only a fraction of bytes.

C. JPEG Structure and DCT Sensitivity

JPEG images use Discrete Cosine Transform (DCT) coding applied to 8×8 pixel blocks [6]. The encoding pipeline proceeds as follows:

- 1) **Color transform.** RGB to YCbCr; chroma channels subsampled (4:2:0 or 4:2:2).
- 2) **DCT per block.** Each 8×8 luma or chroma block produces 64 frequency coefficients arranged in zigzag order.
- 3) **Quantisation.** Coefficients divided by a quality-dependent quantisation table; most AC coefficients are zeroed.
- 4) **Huffman coding.** DC differences between adjacent blocks are coded with a DC Huffman table; remaining AC run-length codes use an AC Huffman table. Both tables are stored in the JPEG header (DHT markers).
- 5) **Bitstream packing.** Huffman-coded symbols are packed into a byte stream beginning with SOI (0xFF 0xD8) and ending with EOI (0xFF 0xD9).

The DC coefficient (spectral index 0) captures the average brightness of each block and has the highest per-coefficient visual impact: corrupting the DC coefficient of one block propagates a visible brightness shift across the entire 8×8 region. A selective encryption scheme targeting only DC coefficients could in principle disrupt the image with minimal ciphertext.

However, DC coefficient positions in the JPEG bitstream are determined by the variable-length Huffman codes preceding them — they do not occur at fixed byte offsets. Locating them requires streaming through the Huffman-coded data, which implies implementing a JPEG decoder on the embedded node. Our stride-based approximation (Section III-B) addresses this constraint.

D. Fragment-Chained MAC: Conceptual Foundation

A Merkle tree [7] commits to an ordered set of leaves; any modification of a leaf or its position is detectable at the root. A fragment-chained MAC is the streaming, append-only analogue: each node in the chain commits to all prior nodes, so the final tag is a commitment to the complete transmission history in order.

Formally, let H be a collision-resistant PRF. Define:

$$h_0 = H(K, \text{context}) \quad (1)$$

$$h_i = H(K, m_i \parallel h_{i-1}), \quad i \geq 1 \quad (2)$$

where m_i is the i -th fragment payload and K is a shared key. Any adversary who modifies m_j must forge h_j ; since h_j is bound to h_{j-1} , forging h_j requires either knowing K or finding a collision in H . Furthermore, inserting a foreign fragment m'_j (from a different session or position) changes h_j , propagating the mismatch to all subsequent tags. The receiver can therefore verify not just individual packet authenticity but also the integrity of the transmission *order and completeness*.

III. SYSTEM DESIGN

A. Threat Model

The FC-HMAC system operates under the following threat model, which is formally encoded in the Tamarin model (Section IV-A).

Network model. All LoRa transmissions are broadcast and receivable by any node in range. The adversary controls the network channel in the Dolev-Yao sense: it can intercept any transmitted fragment, store it indefinitely, and re-inject stored messages at any future time.

Attacker capabilities. The adversary:

- Can intercept, delay, reorder, and replay any LoRa packet.
- Can inject new packets with arbitrary byte content.
- Knows the encryption algorithm, MAC algorithm, stride offsets, and packet format (Kerckhoffs's principle).
- Does *not* know the session key K (pre-shared out-of-band).
- Cannot break HMAC-SHA256 computationally (PRF assumption).

Goals protected against. Table I lists the threats addressed by FC-HMAC:

TABLE I
THREAT MODEL AND FC-HMAC PROTECTION MAPPING

Threat	Attack vector	FC-HMAC protection
Fragment forgery	Inject crafted fragment	F1: tag forgery requires K
Fragment replay (same session)	Re-inject captured fragment	Duplicate frag_id rejected
Cross-session replay	Replay fragments from old session	F3: chain root binds session
Fragment reordering	Deliver frags out of order	Chain tag mismatch at re-ordered position
Silent gap (drop + no detect)	Drop frag i , continue chain	F2: chain break at gap position
Image eavesdropping	Intercept and decode image	C1: JPEG decode FAIL (E19-PSNR)
Key compromise	Brute-force or side-channel K	<i>Out of scope</i>
Physical capture of node	Extract K from flash	<i>Out of scope (hardware TEE)</i>

Goals not protected. The scheme does not provide confidentiality against an adversary who knows K (key management is out of scope). It does not provide availability:

an adversary can jam the LoRa channel or drop all packets. Physical security of the pre-shared key requires hardware trust anchors (secure element, flash encryption) not present in the current EoRa PI prototype.

B. Stride-Based Selective Encryption

1) *Design Rationale and Scope*: True selective encryption targeting DCT DC coefficients requires a Huffman-stream decoder to locate variable-length-coded DC bytes at runtime. On a 240-MHz ESP32-S3 constrained to handle radio interrupts in real time, an inline JPEG decoder would itself introduce CPU overhead comparable to full-image encryption.

We therefore adopt a *structure-approximating* approach. The key observation is that a JPEG file begins with mandatory format structures in the first several hundred bytes:

- Bytes 0–1: SOI marker (0xFF 0xD8)
- Bytes 2–20: APP0/JFIF marker (0xFF 0xE0) and header
- Bytes 21–~200: Quantisation tables (DQT markers, 0xFF 0xDB)
- Bytes ~200–~500: Huffman tables (DHT markers, 0xFF 0xC4)
- Bytes ~500+: Start of scan (SOS marker, 0xFF 0xDA) and encoded data

Corrupting any of the quantisation or Huffman table entries renders the JPEG bitstream undecodable, because every JPEG decoder uses these tables to interpret the subsequent AC/DC runs. A stride starting at byte 0 with 16-byte windows every 64 bytes will intersect these critical structures during its first several iterations.

The contribution of the stride-based approach is therefore correctly framed as *lightweight visual obfuscation achieving format-level disruption*, not frequency-domain selective encryption. The actual overlap between stride positions and true DCT DC coefficient byte positions remains to be quantified (see Section VII-A).

Algorithm 1 Stride-Based Selective Encryption (ENC_DC_ONLY mode)

Require: Image byte array $P[0 \dots N-1]$, session key K , frag counter c

Ensure: Encrypted byte array C

```

1:  $C \leftarrow \text{copy}(P)$ 
2:  $\text{stride} \leftarrow 64$ ;  $\text{wlen} \leftarrow 16$ 
3: for  $i \leftarrow 0$  to  $N-1$  step  $\text{stride}$  do
4:    $ks[0:16] \leftarrow \text{HMAC-SHA256}(K, \langle i \parallel c \rangle)[16]$ 
5:   for  $j \leftarrow 0$  to  $\min(\text{wlen}-1, N-i-1)$  do
6:      $C[i+j] \leftarrow P[i+j] \oplus ks[j]$ 
7:   end for
8: end for
9: return  $C$ 
```

2) *Algorithm Specification*: The frag counter c differentiates keystreams across repeated calls for the same image buffer. For a 2,046-byte JPEG, the loop executes at strides $i \in \{0, 64, 128, 192, 256, 320, 384, 448, 512\}$, producing 9 keystream calls and encrypting up to $\min(16, N-i)$ bytes per stride. Hardware measurement (E19) confirms 496

of 2,046 bytes encrypted (24.2%), at a CPU cost of 2,807 μs — a 76% reduction compared to full-image encryption estimated at $\approx 11,600 \mu\text{s}$.

3) *Three Encryption Modes*: The firmware implements three modes selectable at compile time via the ENC_MODE build flag:

TABLE II
ENCRIPTION MODE COMPARISON (2,046-BYTE JPEG, SF9/BW125)

Mode	Packets	Airtime	CPU enc (μs)	Enc bytes
ENC_NONE	9	14.9 s	0	0%
ENC_DC_ONLY	9	14.9 s	2,807	24.2%
ENC_FULL	9	14.9 s	$\approx 11,600$	100%

A critical design point: all three modes transmit the *same number of packets at the same airtime*, because packet count is determined by $\lceil N/233 \rceil$ regardless of encryption mode. The efficiency gain of ENC_DC_ONLY is therefore in *CPU*, not in airtime or duty-cycle consumption. This distinction is explored further in Section V-C.

4) *LoRa Airtime Model*: For completeness, we derive the airtime formula used throughout this paper. Using the standard LoRa time-on-air model [2], with spreading factor $SF = 9$, bandwidth $BW = 125 \text{ kHz}$, coding rate $CR = 4/7$, explicit header mode, and CRC enabled:

$$T_s = \frac{2^{SF}}{BW} = \frac{512}{125000} = 4.096 \text{ ms} \quad (3)$$

$$N_{\text{preamble}} = 8 \text{ symbols (default)}$$

$$T_{\text{preamble}} = (N_{\text{preamble}} + 4.25) \times T_s = 50.9 \text{ ms} \quad (4)$$

For a 248-byte payload packet (7 header + 233 data + 8 HMAC):

$$N_{\text{sym,payload}} = 8 + \max\left(\left\lceil \frac{8 \times 248 - 4 \times SF + 28 + 16}{4 \times (SF - 2)} \right\rceil \times CR, 0\right) = 107 \quad (5)$$

$$T_{\text{packet}} = T_{\text{preamble}} + N_{\text{sym,payload}} \times T_s = 50.9 + 107 \times 4.096 \approx 489 \text{ ms} \quad (6)$$

The hardware-measured airtime of 1,692 ms per fragment includes inter-packet gaps, radio mode-switch overhead, and SX1262 TX-to-RX turnaround time. The theoretical packet-on-air time (489 ms) is consistent with the fraction of 1,692 ms spent with the antenna active. The total image airtime is:

$$T_{\text{image}} = N_f \times T_{\text{frag}} = 9 \times 1,692 \approx 14.9 \text{ s} \quad (7)$$

matching the hardware measurement in Table X.

5) *Security Claim and Honest Framing*: We claim that ENC_DC_ONLY achieves *visual-domain confidentiality* under the following attacker model: the adversary intercepts all LoRa packets, reassembles the fragment stream, and attempts JPEG decoding. The stride corrupts the SOI marker and Huffman tables, preventing decoding. The claim is *not* that the scheme achieves IND-CPA security or hides frequency-domain coefficients selectively — it does not. A determined cryptanalyst with knowledge of the stride pattern and access to many plaintext-ciphertext pairs could recover the keystream (since it is static per-stride for a fixed image and frag

counter), especially if the plaintext header bytes are known. The scheme's purpose is operational security against passive image reconstruction, not semantic security.

C. Fragment-Chained HMAC (FC-HMAC)

1) *Problem Statement and Threat Model*: A 2,046-byte JPEG image spans 9 LoRa fragments. The threat model for the fragmentation layer is a Dolev-Yao adversary [4] who:

- Can intercept any LoRa packet transmitted in range.
- Can delay, replay, or reorder intercepted packets in any session.
- Can attempt to inject forged fragments with a chosen payload.
- Does *not* know the session key K (pre-shared, not on the radio).

Per-packet independent HMAC prevents the attacker from forging *individual* fragments but provides no protection against:

- **Replay attack**: fragments from session s replayed in session s' ; each fragment passes its HMAC check independently.
- **Ordering attack**: fragment i replaces fragment j ; both have valid MACs, and the receiver has no way to detect the swap.
- **Cross-splice**: fragments from two sessions spliced together; each fragment is individually valid.

FC-HMAC addresses all three by binding each fragment's tag to the complete prior chain state.

2) *FC-HMAC Construction*: Let K be a 16-byte pre-shared session key. Let N_f be the total number of fragments for one image. Define the *chain root* as:

$$h_0 = \text{HMAC-SHA256}(K, 0x00000000 \parallel N_f)[8] \quad (8)$$

For each fragment $i = 1, \dots, N_f$, compute the *chain tag*:

$$tag_i = \text{HMAC-SHA256}\left(K, \underbrace{frag_id_i}_{2B} \parallel \underbrace{flags_i}_{1B} \parallel \underbrace{payload_i}_{\text{up to 233B}} \parallel \underbrace{tag_{i-1}}_{8B}\right)[8] \quad (9)$$

The receiver maintains a running chain state $\hat{h}$ initialised to h_0 . Upon receiving fragment i , it recomputes $\hat{t}_i$ from $\hat{h}$ and the received fields, then checks $\hat{t}_i \stackrel{?}{=} tag_i$. If the check passes, $\hat{h} \leftarrow tag_i$. If it fails, the chain is broken at position i , and all subsequent fragments cannot be chain-verified (though they may be buffered for potential retransmit-based recovery).

Why chaining achieves replay resistance. An adversary who replays fragments from session s in session s' must forge tags consistent with the chain state of session s' . The chain state depends on $h_0^{(s')}$, which depends on session s' 's unique context (when `img_seq` is included in the root — see Section VII-B). Without knowing K , the adversary cannot compute consistent $tag_i^{(s')}$ values.

Why chaining achieves ordering resistance. Swapping fragment j into position $i \neq j$ changes `frag_id` and inserts a different payload into the HMAC input at position i . The recomputed tag will not match the transmitted tag_i , which was computed with the correct `frag_id = i` and its own payload.

3) *Chain Initialisation and img_seq*: Equation (8) uses a zero nonce, which is the implementation in the E19 firmware. A production-ready implementation should include a session-unique image sequence number `img_seq` to bind the chain root to a specific session:

$$h_0^{(\text{prod})} = \text{HMAC-SHA256}(K, \text{img_seq} \parallel N_f)[8] \quad (10)$$

The Tamarin formal model (Section IV-A) uses Equation (10), and the F3 (No_Replay) lemma is proved under this stronger construction. The E19 firmware gap (omitting `img_seq`) is documented as a known limitation in Section VII-B.

4) *Security Parameters and Truncation Analysis*: The HMAC output is truncated to 8 bytes (64 bits). Table III analyses the security-overhead tradeoff for three candidate truncation lengths:

TABLE III
HMAC TRUNCATION LENGTH: SECURITY VS. OVERHEAD TRADEOFF

Tag size	Overhead	Forgery prob.	Assessment
4 B (32-bit)	1.7%	2^{-32}	Insufficient
8 B (64-bit)	3.3%	2^{-64}	Target (selected)
12 B (96-bit)	5.0%	2^{-96}	Strong but wasteful

Under LoRa duty-cycle constraints, a single adversarial node can inject at most ≈ 100 packets per hour. At this injection rate, exhaustive search against a 64-bit tag would require $\approx 1.8 \times 10^{17}$ years in expectation — well beyond any practical threat.

D. Packet Format and Frame Layout

Each LoRa packet transmitted by the FC-HMAC protocol carries the following fixed-layout frame:

TABLE IV
FC-HMAC PACKET FRAME LAYOUT

Field	Width	Description
<code>magic</code>	2 B	Framing bytes: 0xE1 0x9A
<code>fragment_id</code>	2 B	Zero-indexed fragment number (0 to $N_f - 1$)
<code>total_frags</code>	2 B	Total fragments in this image
<code>flags</code>	1 B	Bit 0: last fragment; bits 1–2: enc mode
<code>payload</code>	N B	Possibly encrypted image bytes
<code>hmac8</code>	8 B	8-byte truncated chained HMAC tag
Max total	$7 + N + 8 \leq 255 \text{ B}; N \leq 240 \text{ B}$	
Used ($N = 233$)	248 B total per packet	

The 7-byte fixed header plus 8-byte HMAC tag consume 15 bytes, leaving 240 B for payload. In practice the E19 firmware uses $N = 233$ bytes payload for a 248-byte total, providing a 7-byte margin below the 255-byte hardware limit. The 8-byte `hmac8` field represents 3.3% overhead relative to the 241-byte application-layer frame (7 header + 234 B payload before HMAC).

The `magic` bytes serve as a lightweight frame synchronisation marker: the receiver discards any packet that does not begin with 0xE1 0x9A, preventing stray LoRa broadcasts from polluting the fragment reassembly buffer.

1) *Fragment Reassembly at the Receiver*: The receiver maintains the following state per ongoing image transmission:

- `total_frags`: total fragment count, learned from the first received fragment's `total_frags` field.
- `recv_bitmap`: 64-bit bitmask (supports up to 64 fragments) indicating which `fragment_id` values have been received.
- `chain_state` ($\hat{h}$): the 8-byte chain tag of the last successfully chain-verified fragment. Initialised to h_0 (Equation 8) before the first fragment.
- `reassembly_buffer`: array of N_f payload slots, each 233 B.
- `chain_broken`: boolean flag set when a chain verification failure is detected; once set, subsequent fragments cannot be chain-verified but are still stored for diagnostic purposes.

Algorithm 2 gives the per-fragment receive procedure:

Algorithm 2 FC-HMAC Fragment Reception and Chain Verification

Require: Received packet bytes pkt

```

1: Parse magic, frag_id, total_frags, flags,
   payload, hmac8
2: if magic  $\neq$  0xE19A then return DISCARD
3: end if
4: if frag_id  $\geq$  total_frags then return DISCARD
5: end if
6:  $\hat{t} \leftarrow \text{HMAC-SHA256}(K, \langle \text{frag\_id}, \text{flags}, \text{payload}, \hat{h} \rangle)$ 
7: if  $\hat{t} \neq \text{hmac8}$  then
8:   Set chain_broken  $\leftarrow$  true
9:   Record chain-break at frag_id; return
   CHAIN_BREAK
10: end if
11:  $\hat{h} \leftarrow \text{hmac8}$  ▷ Advance chain state
12: Copy payload to reassembly_buffer[frag_id]
13: Set bit frag_id in recv_bitmap
14: if recv_bitmap =  $(1 \ll \text{total\_frags}) - 1$  then
15:   Deliver assembled image to application return COMPLETE
16: end if return OK

```

When all bits in `recv_bitmap` are set, the image is declared complete and passed to the application layer. A missing fragment (due to channel loss) breaks the chain from that position onward; the receiver can request a retransmit using the bitmap information (in an ACK-capable variant) or accept a partial image by re-anchoring the chain at the next received fragment.

The `chain_broken` flag is important for diagnostics: it allows the receiver to distinguish between (a) a complete image received without chain breaks, (b) a complete image received with at least one chain break (indicating tampering or a firmware bug), and (c) an incomplete image (missing fragments). In the E19 experiment, case (a) was observed for all 8 transmitted images.

IV. SECURITY ANALYSIS

A. Tamarin Formal Model

1) *Model Overview*: We model FC-HMAC in Tamarin Prover [4] (version 1.13.0, executed in the `lora-mesh-tamarin` Docker container). The model file is `phase2_tamarin_models/paper_d/fc_hmac.spthy` (verification log: `phase2_tamarin_models/paper_d/fc_hmac_v`). The model encodes a two-fragment session to keep the state space finite while capturing all relevant security properties.

2) *Modelling Decisions*: **MAC as perfect PRF**. The `mac/2` function symbol is declared with no inversion or cancellation equations. This models HMAC-SHA256 as an ideal pseudorandom function under the standard symbolic (Dolev-Yao) model, where the adversary cannot invert, decompose, or partially evaluate the MAC without the key.

Dolev-Yao network adversary. The attacker controls all network communication: it can intercept, store, replay, and inject arbitrary LoRa messages. It knows all public values (fragment IDs, total-frag counts, ciphertexts) but not the session key K .

State-space bounds. Two Tamarin restrictions limit rule applications: `OneSetup` ensures a single sender initialisation per session, and `ReceiverOnce` bounds receiver state to one verification chain per session. These restrictions prevent unbounded rule application while preserving the security properties of interest.

Chain initialisation. The `Sender_Init` rule computes $h_0 = \text{mac}(K, \langle \text{sess}, \text{tf} \rangle)$, where `sess` is a fresh session identifier and `tf` is the total fragment count. This matches Equation (10).

Fragment rules. `Sender_Frag1` and `Sender_Frag2` each compute a chained tag $\text{tag}_i = \text{mac}(K, \langle \text{frag_id}, f, p, \text{tag}_{\text{prev}} \rangle)$ and output the packet to the adversary-controlled network via `Out(...)`. `Receiver_Frag1` and `Receiver_Frag2` each receive a packet via `In(...)`, re-derive the expected tag, and enforce equality via the `Eq(lhs, rhs)` restriction, which causes Tamarin to prune all execution traces where the check fails.

3) *Lemma Statements and Results*: The four lemmas and their proof results are summarised in Table V. All four lemmas were proved in 1.53 seconds total on a standard workstation (Intel Core i7, 16 GB RAM) running Tamarin 1.13.0.

TABLE V
TAMARIN VERIFICATION RESULTS (FC_HMAC.SPTHY, 1.53 S TOTAL)

ID	Lemma Name	Result	Steps
F4	<i>Executability</i>	PROVED	14
F1	<i>Fragment_Authenticity</i>	PROVED	10
F2	<i>Chain_Gap_Detectable</i>	PROVED	3
F3	<i>No_Replay</i>	PROVED	12

F4 (Executability, 14 steps) is a sanity lemma: it proves the existence of a complete honest execution trace where the sender sends two fragments and the receiver successfully verifies both. This confirms the model is not vacuously satisfied.

F1 (Fragment_Authenticity, 10 steps) states: if the receiver successfully verifies fragment i , then there exists a

sender action that emitted a fragment with the same fragment ID, flags, payload, and chain tag. Formally, the adversary cannot cause the receiver to accept a fragment that was never sent by a legitimate sender holding key K .

F2 (Chain_Gap_Detectable, 3 steps) states: if fragment i is missing from the chain and fragment $i + 1$ is received, then the receiver’s verification of fragment $i + 1$ will fail. The chain cannot “skip” a missing fragment without detection. This property is unique to FC-HMAC and has no analogue in per-packet independent HMAC schemes.

F3 (No_Replay, 12 steps) states: if the receiver accepts fragment i from session s' , then it was originally sent in session s' (not replayed from a different session $s \neq s'$). This property requires `img_seq` in the chain root, as discussed in Section VII-B.

4) *Security Properties Summary*: Table VI summarises the four security properties in the FC-HMAC model and maps them to the design elements and attacker goals they address.

TABLE VI
FC-HMAC SECURITY PROPERTIES AND DESIGN MAPPING

Lemma	Property	Precondition	Defeated attack
F1	Fragment authenticity	Key K unknown to adversary	Fragment forgery
F2	Chain-gap detectable	Chain integrity	Silent fragment drop
F3	No cross-session replay	<code>img_seq</code> in chain root	Session replay
F4	Executability (witness)	None	Protocol divergence

5) *Comparison with Independent Per-Packet HMAC*: To underscore the novelty of chain-based verification, Table VII contrasts the security properties achievable by independent per-packet HMAC (as in Ryzek [3]) versus FC-HMAC.

TABLE VII
INDEPENDENT PER-PACKET HMAC VS. FC-HMAC: SECURITY PROPERTY COVERAGE

Security Property	Per-Packet HMAC	FC-HMAC
Fragment forgery prevention	✓	✓
Individual packet integrity	✓	✓
Fragment reorder detection	×	✓
Fragment insertion detection	×	✓
Cross-session replay prevention	×	✓ (with <code>img_seq</code>)
Silent gap detection	×	✓
Formally verified (machine-checked)	×	✓

6) *F3 Firmware Gap*: The Tamarin F3 lemma is proved under the model with `img_seq` in the chain root. The E19 firmware (Section V-B) uses a zero nonce instead, creating a gap between the formal model and the hardware implementation. This is a documented limitation; the E20 firmware (planned) addresses it by encoding `img_seq` in bytes 0–3 of the `frag_id=0` payload prefix. The F1, F2, and F4 lemmas hold regardless of whether `img_seq` is present.

B. Visual Security Evaluation (E19-PSNR)

1) *Evaluation Methodology*: The visual security evaluation script (`tools/psnr_attack_eval.py`) applies each encryption mode to a synthetic JPEG and then simulates attacker reconstruction attempts. Two attacker models are considered:

A1 — Naive attacker: receives the encrypted byte stream (after FC-HMAC verification strips the header) and attempts to decode it directly as a JPEG file using a standard decoder.

A2 — Format-aware attacker: knows the stride pattern (16 bytes per 64-byte window) and the starting offset. Before decoding, the attacker zeroes all bytes at stride positions, replacing ciphertext with zeros to recover the underlying format structures. This simulates a partial chosen-plaintext attack by an adversary with full knowledge of the encryption algorithm but not the key.

For both attackers, success is measured as either a finite PSNR value (image decoded, compared to original) or a JPEG decode failure.

TABLE VIII
VISUAL SECURITY EVALUATION RESULTS (E19-PSNR, `PSNR_ATTACK_EVAL.PY`)

Mode	Enc bytes	Stride calls	A1	A2
ENC_NONE	0%	0	PSNR = ∞	N/A
ENC_DC_ONLY	24.9%	9	Decode FAIL	Decode FAIL
ENC_FULL	99.8%	256	Decode FAIL	Decode FAIL

2) *Results and Analysis*: ENC_DC_ONLY at 24.9% encrypted bytes completely prevents JPEG reconstruction under both attacker models. The A2 format-aware attacker fails because the stride begins at byte 0 (SOI marker) and the 9 encryption windows cover the first $9 \times 64 = 576$ bytes of the image. Within this range, the JPEG SOI marker (bytes 0–1), APP0 header (bytes 2–20), quantisation tables (bytes ~21–200), and Huffman tables (bytes ~200–500) are all partially encrypted. Even if the attacker zeroes the encrypted positions, the non-zero residual pattern in the surrounding bytes is inconsistent with any valid JPEG structure, causing decoder failure.

The PSNR target originally set for this evaluation (PSNR attack < 20 dB) is superseded: decode failure provides a stronger guarantee than a finite PSNR value, since the attacker obtains zero visual information rather than a degraded reconstruction.

3) *Limitation: Semantic Security*: The E19-PSNR evaluation confirms that format-level obfuscation is achieved against both attacker models tested. However, we do not claim semantic security (IND-CPA) for the stride scheme. An adversary with access to multiple plaintexts encrypted under the same key and stride would observe the same XOR pattern at stride positions across images, potentially recovering keystream blocks. The scheme is intended for deployments where the same key is rotated periodically and the attacker has limited oracle access.

V. HARDWARE EVALUATION

A. Platform and Experimental Setup

Both transmitter and receiver nodes use the EoRa PI development board, which integrates an Espressif ESP32-S3 [8] (Xtensa LX7 dual-core, 240 MHz, 512 KB internal SRAM, 2 MB PSRAM) with a Semtech SX1262 LoRa transceiver [2] on-board. The SX1262 is connected via SPI with the following

pin mapping: NSS = GPIO7, SCK = GPIO5, MOSI = GPIO6, MISO = GPIO3, RST = GPIO8, BUSY = GPIO34, DIO1 = GPIO33.

The firmware is compiled with ESP32 Arduino SDK 3.x with CDCOnBoot enabled for USB serial output. The mbedTLS library bundled with the SDK provides software-only HMAC-SHA256 and AES operations; no hardware cryptographic accelerator is used. All CPU timing measurements reflect pure software execution on a single Xtensa LX7 core.

Experimental parameters are summarised in Table IX.

TABLE IX
HARDWARE PLATFORM AND LoRa CONFIGURATION (E19)

Parameter	Value
MCU	ESP32-S3 (Xtensa LX7, 240 MHz, 2 MB PSRAM)
Radio	Semtech SX1262
Firmware SDK	ESP32 Arduino SDK 3.x, mbedTLS
Frequency	922.0 MHz
Spreading Factor	SF9
Bandwidth	125 kHz
Coding Rate	4/7
TX power	2 dBm
Max LoRa payload	255 B (SX1262 hardware register limit)
Fragment size	248 B (7 B header + 233 B data + 8 B HMAC)
Session key	16-byte pre-shared (AES-128 equivalent)
Test image	Synthetic JPEG, 200 × 150 px, 2,046 B
Fragments per image	9
Measured fragment airtime	1,692 ms
Measured image airtime	14.9 s
Experiment duration	120 s
Images transmitted	8 (complete)

The transmitter sends the test image as a C byte array (`test_image.h`) embedded in the firmware flash. This approach isolates the protocol layer from camera hardware variability, enabling reproducible measurements. A live camera attachment (OV2640, DVP interface via EoRa PI GPIO) is planned for Phase 3 validation (Section VII-C).

B. E19 Experiment Results

Table X reports all primary measurements from experiment E19 (results file: `hardware_experiments/E19_selective_enc/results/e19_results.md`).

TABLE X
E19 HARDWARE EXPERIMENT SUMMARY (120 s, 2 × EoRa PI, SF9/BW125)

Metric	Measured Value
Images transmitted	8
Total fragments	72
Fragments verified (chain_ok)	72 / 72 = 100%
Packet Delivery Ratio (PDR)	100%
Airtime per image	14.9 s
Measured fragment airtime	1,692 ms per fragment
HMAC overhead (per packet)	8 B / 241 B = 3.3%
HMAC overhead (per image)	72 B / 2,046 B = 3.5%
HMAC CPU per image	≈950 μs (9 HMAC-SHA256 calls)
Enc CPU per image (DC_ONLY)	2,807 μs (9 HMAC-PRF keystream calls)
Total crypto CPU per image	950 + 2,807 = 3,757 μs (≈ 3.76 ms)
Encrypted bytes	496 / 2,046 B = 24.2%
Encryption mode	ENC_DC_ONLY

All 72 fragments in the 120-second run pass FC-HMAC chain verification under benign conditions (no packet loss, no tampering, line-of-sight 1-metre indoor range). The 100% chain-ok rate confirms that (i) the HMAC-SHA256 computation is deterministic and consistent between sender and receiver, (ii) the chain initialisation and tag update logic are correctly implemented, and (iii) the SX1262 LoRa radio delivers fragments without bit errors in this test environment.

The HMAC overhead of 3.3% per packet is below the theoretical 4-byte-tag bound of 1.7%, which would be insufficient for security, and below the 12-byte bound of 5.0%, which would be wasteful — placing the 8-byte design squarely at the optimal security-overhead point (Table III).

L) *Fragment Timing and Inter-Fragment Gap*: Table XI breaks down the measured per-fragment timing into its constituent phases as observed in the E19 firmware.

TABLE XI
PER-FRAGMENT TIMING BREAKDOWN (E19, 248-BYTE FRAGMENT, SF9/BW125)

Phase	Duration
SX1262 TX preparation (SPI config)	≈5 ms
LoRa preamble + header on air	50.9 ms
Payload on air (107 symbols × 4.096 ms)	438 ms
TX-to-idle turnaround	≈2 ms
<i>Theoretical packet on air</i>	<i>489 ms</i>
Application-layer inter-fragment gap	≈1,200 ms
Total per-fragment cycle (measured)	1,692 ms

The 1,200 ms inter-fragment gap is enforced at the application layer to comply with the ETSI 1% duty-cycle rule: $T_{\text{on-air}} / (T_{\text{on-air}} + T_{\text{gap}}) = 489 / (489 + 1,200) \approx 0.29\%$, well below the 1% ceiling. The gap also gives the receiver time to process the received fragment and update its chain state before the next packet arrives.

Within the 1,200-ms inter-fragment gap, the sender computes the selective encryption keystream and the FC-HMAC tag for the next fragment ($3,757 \mu\text{s} / 9 = 417 \mu\text{s}$ per fragment), builds the packet frame, and loads it into the SX1262 FIFO. The crypto processing occupies only 0.035% of the inter-fragment gap, confirming that FC-HMAC introduces no detectable increase in duty-cycle compliance risk.

C. Efficiency Analysis

1) *Airtime Invariance Across Encryption Modes*: A fundamental observation from Table II is that selective encryption does *not* reduce airtime relative to full encryption. All three modes (ENC_NONE, ENC_DC_ONLY, ENC_FULL) transmit 9 fragments of 248 bytes each, producing identical 14.9-second airtime.

This invariance arises because airtime is determined by:

$$T_{\text{image}} = N_f \times T_{\text{frag}}(L_{\text{frag}}) \quad (11)$$

where $N_f = \lceil N_{\text{img}} / N_{\text{payload}} \rceil$ depends only on image size and per-fragment payload capacity, and T_{frag} depends only on fragment length and LoRa parameters. Neither depends on how many bytes within the payload are encrypted. Encryption changes individual byte values but not the byte count.

The practical implication for the design: selective encryption is *not* an airtime optimisation technique. It is a CPU optimisation technique. Claims in the literature that selective encryption reduces LoRa duty-cycle consumption should be scrutinised to determine whether they refer to CPU cost or to selective *fragment omission* (transmitting fewer fragments by skipping non-critical image regions).

TABLE XII
CPU COST BY ENCRYPTION MODE (2,046-BYTE IMAGE, ESP32-S3
240 MHz)

Mode	Enc CPU	HMAC CPU	Total crypto	vs. FULL
ENC_NONE	0 μ s	950 μ s	950 μ s	−92%
ENC_DC_ONLY	2,807 μ s	950 μ s	3,757 μ s	−76%
ENC_FULL	≈11,600 μ s	950 μ s	≈12,550 μ s	baseline

2) *CPU Cost Breakdown*: The ENC_DC_ONLY mode reduces encryption CPU by 76% compared to ENC_FULL (2,807 μ s vs. ≈11,600 μ s) while maintaining format-level visual obfuscation (Table VIII). The HMAC CPU (950 μ s) is identical across all modes because all fragments carry the FC-HMAC chain tag.

Total crypto CPU for ENC_DC_ONLY is 3,757 μ s (3.76 ms) per 2,046-byte image. For a one-image-per-minute transmission schedule, the image transmission occupies 14.9 s of airtime and 3.76 ms of crypto CPU per 60-second cycle. The crypto fraction of the non-airtime processing budget is therefore negligible: $3.76 \text{ ms} / (60 \text{ s} - 14.9 \text{ s}) \approx 0.008\%$.

3) *Overhead Scalability*: The FC-HMAC overhead has two components:

- 1) **Per-packet tag overhead**: 8 B per fragment, constant regardless of payload size. For a 233-byte payload, this is 3.3%. For a 30-byte payload (in a network where packets are smaller), this rises to 21% — a significant penalty. The 8-byte tag is therefore best suited to near-maximum-payload fragments.
- 2) **Per-image CPU overhead**: $O(N_f)$ HMAC calls, one per fragment. For a 20-kB image (as in Ryzek [3]) split into $\lceil 20480/233 \rceil = 88$ fragments, the HMAC CPU scales to ≈9,250 μ s — still below the estimated 113,600 μ s for full encryption of 20 kB, representing the same 76% relative saving.

VI. RELATED WORK

A. Image Transmission Security over LoRa

Ryzek and Sobieraj [3] present the most closely related prior work: an ESP32-CAM paired with an E220 LoRa module (SX1262-based) transmitting 20 kB camera images with full AES-128-CBC encryption and independent per-packet HMAC-SHA256. They report PDR = 100% in indoor experiments and <1% energy overhead on the transmitter side for encrypted vs. plaintext transmission. Their work demonstrates the viability of application-layer encryption on ESP32-class hardware.

Our work differs in three structural ways. First, we use *selective* rather than full encryption, demonstrating a 76% CPU saving while maintaining visual obfuscation. Second,

we replace independent per-packet HMAC with a *chained* structure (FC-HMAC) that binds the entire fragment sequence, enabling detection of ordering attacks and cross-session replay. Third, we provide machine-checked formal proofs of chain-gap detectability and no-replay via Tamarin Prover — properties that per-packet HMAC schemes cannot satisfy and that have not been formally verified in prior LoRa image security work.

B. Selective Encryption for Compressed Media

Selective encryption has been extensively studied for MPEG video [9]: encrypting I-frame headers, motion vectors, or DC coefficients provides visual confidentiality at a fraction of full-encryption cost. The key insight is that video and image codecs have an *entropy hierarchy*: a small number of structurally critical bits, when corrupted, propagate errors throughout the decoded signal. Targeting these bits maximises disruption per encrypted byte.

Applying this principle to JPEG over LoRa introduces a new constraint absent from video settings: the encoder runs on a microcontroller without floating-point DSP hardware, the JPEG bitstream is static (not a live stream), and the per-fragment overhead budget is measured in bytes, not kilobytes. Our stride-based approximation is a pragmatic adaptation that achieves the same format-level disruption without requiring a Huffman-stream parser.

Earlier work on selective JPEG encryption for mobile and embedded devices targeted megapixel-range images on processors with hardware floating-point and large caches, where full JPEG parsing is feasible. At the LoRa constraint level (2 KB images, 240-MHz MCU with no hardware AES, 8-byte overhead budget per packet), a parsing-free approximation is the appropriate design point.

It is important to note that selective encryption of JPEG images can achieve format disruption at the *syntax* layer (as in our scheme) or at the *semantic* layer (targeting specific DCT frequencies). Our scheme operates at the syntax layer: by corrupting the Huffman table segments, we prevent the decoder from interpreting any subsequent coded data. This is a weaker claim than frequency-domain selective encryption but is achievable with a 9-call keystream computation in firmware. The relationship between syntax-layer and frequency-domain selective encryption is discussed further in Section VII-A.

C. MAC Authentication for Constrained Wireless Networks

TinySec [10] introduced MAC-only and encryption+MAC modes for IEEE 802.15.4 sensor networks in 2004, demonstrating that 4-byte MACs are sufficient for low-data-rate wireless links at acceptable energy overhead. TinySec uses CBC-MAC (cipher-block chaining MAC) applied independently to each packet, which prevents forgery of individual packets but does not chain across packet boundaries. TinySec’s design inspired the choice of compact MAC sizes in constrained networks; our 8-byte tag selection extends this by adding cross-packet chain binding.

RFC 3610 [9] defines AES-CCM, which combines encryption and authentication in a single pass using CTR

mode for encryption and CBC-MAC for authentication. AES-CCM requires hardware AES support (or significant software overhead) and does not natively support fragment chaining. Furthermore, AES-CCM is not available in mbedTLS without an AES hardware accelerator on ESP32-S3 (the hardware AES driver is not linked in the ESP32 Arduino SDK configuration used in E19). Our scheme uses HMAC-SHA256 (available as a pure-software primitive in mbedTLS), truncated to 8 bytes, chained across fragments.

The HMAC-based PRF construction [7] provides a provable security reduction from the MAC security to the PRF security of the underlying hash function (SHA-256), under the random oracle model. This is a stronger foundation than CBC-MAC (which has known weaknesses for variable-length messages) and more widely available in embedded TLS stacks than AES-CCMAC.

D. Formal Verification of IoT Protocol Security

Tamarin Prover [4] has been applied to verify LoRaWAN join-accept procedures [4], lightweight RFID authentication protocols, and TLS 1.3. In the LoRa context, prior Tamarin work has focused on the LoRaWAN session-establishment phase, not on application-layer fragmentation integrity. Our FC-HMAC model is the first formal verification of a fragment-chained MAC for image transmission over LoRa.

The F2 (Chain_Gap_Detectable) and F3 (No_Replay) lemmas are novel contributions: F2 captures a property that is unique to chained MACs and has no analogue in per-packet HMAC systems; F3 proves session binding that is typically assumed but not formally checked in IoT security papers.

E. Detailed Comparison: FC-HMAC vs. Ryzek & Sobieraj 2025

Table XIII provides a structured seven-dimension comparison between FC-HMAC (this work) and the nearest prior work [3].

VII. DISCUSSION

A. Limitation B1: Stride Approximation vs. True DCT-Domain Encryption

The primary theoretical limitation of Phase 1 is that the stride-based scheme is *not* true frequency-domain selective encryption. We explicitly acknowledge and reframe this as follows.

True selective encryption for JPEG — specifically targeting DCT DC coefficient bytes — requires streaming through the Huffman-coded bitstream, which encodes DC differences as variable-length codes that do not occur at predictable byte offsets. Implementing such a parser on a constrained embedded node introduces CPU overhead comparable to full-image encryption for the decoding step alone.

The stride-based approximation achieves visual-domain disruption through a different mechanism: it corrupts the JPEG format infrastructure (SOI marker, Huffman tables) rather than DCT frequency coefficients. The E19-PSNR evaluation confirms that this achieves the operational goal (JPEG decode

failure under both attacker models) without frequency-domain targeting. The honest framing of the contribution is therefore:

Lightweight visual obfuscation via structure-level disruption, not frequency-domain selective encryption.

Future work (`tools/jpeg_dc_offset_extractor.py`) will quantify the actual overlap between stride positions and true DC coefficient byte positions for a corpus of JPEG images. If the overlap is high (>50%), the approximation claim is partially vindicated; if it is low (<20%), the obfuscation result must be attributed entirely to header corruption, and the “DCT-targeting” framing should be abandoned.

The path to true frequency-domain selective encryption for Phase 2 would require: (a) a lightweight JPEG entropy decoder that identifies DC code boundaries, (b) in-place XOR of those bytes only, and (c) evaluation of whether the decoder overhead is justified by the additional security properties (IND-CPA against an attacker who knows the stride pattern but not the key).

B. Limitation F3: `img_seq` Omission in E19 Firmware

The Tamarin F3 lemma (No_Replay) is proved under the assumption that `img_seq` is included in the chain root (Equation 10). The E19 firmware uses a zero nonce (Equation 8), creating a gap between the formal model and the hardware implementation.

Under the E19 firmware, an adversary who captures a complete valid fragment sequence from session s and replays it as session s' will succeed: the receiver re-derives the same chain tags (since h_0 is the same for both sessions) and accepts the replayed fragments. This attack requires the adversary to capture a complete prior session in its entirety, which is a non-trivial requirement in a LoRa deployment where the attacker may not be continuously present. Nevertheless, it represents a formal gap that should be closed before deployment.

The fix is a one-line firmware change: replace `0x00000000` with `img_seq` (a monotonically incrementing `uint32` counter) in the `Sender_Init` HMAC call. This modification brings the firmware into conformance with the Tamarin model and closes the F3 gap.

C. Future Work

E20 — FC-HMAC verifiability under packet loss ($3\times$ EoRa PI). A three-node experiment (sender, relay/dropper, receiver) will evaluate chain verifiability at 10% and 20% packet drop rates, measuring the verifiable prefix length and comparing FC-HMAC (chained) vs. per-packet HMAC (Ryzek model). The E20 firmware adds the `img_seq` fix above, closing the F3 gap on hardware.

E21 — CPU/energy tradeoff measurement. A controlled comparison of `ENC_NONE` / `ENC_DC_ONLY` / `ENC_FULL` on the same 2,046-byte JPEG, measuring both CPU microseconds and, if a Joulescope power meter is available, milliwatt-hours per image per mode. The `ENC_FULL` CPU estimate of $\approx 11,600 \mu s$ used in this paper is derived from AES-128 software benchmarks on ESP32-S3; E21 will provide a direct hardware measurement.

TABLE XIII
FC-HMAC VS. RYCZEK & SOBIEK 2025: SEVEN-DIMENSION COMPARISON

Dimension	Rycek & Sobieraj (Electronics 2025) [3]	FC-HMAC (this work)
Encryption scope	Full image, AES-128-CBC, 100% bytes encrypted	Stride-based selective (24.2% bytes); format-level obfuscation; 76% CPU saving
Authentication	Per-packet HMAC-SHA256, independent tags; each packet self-contained	FC-HMAC chain: tag_i commits to tag_{i-1} ; re-ordering, insertion, replay detectable
Loss behaviour	Lost packet does not affect other packets; receiver can accept partial image transparently	Chain break at loss position; receiver obtains a verified prefix chain up to the first gap
Formal verification	None	Tamarin Prover: F1 (authenticity), F2 (chain-gap), F3 (no-replay), F4 (executability); all proved in 1.53 s
HMAC overhead	$\approx 4\text{--}5\%$ (estimated from paper data; not directly hardware-measured)	3.3% per packet, 3.5% per image; directly hardware-measured in E19
PSNR under attack	Not measured; visual security assumed from full encryption	JPEG decode FAIL for both A1 (naive) and A2 (format-aware) attackers; measured via <code>psnr_attack_eval.py</code>
Hardware platform	ESP32-CAM + EBYTE E220 (SX1262), 20kB JPEG images, OV2640 camera	EoRa PI (ESP32-S3 + SX1262), 2,046-byte synthetic JPEG; OV2640 attachment planned for Phase 3

Phase 3 live camera validation. The EoRa PI GPIO bank supports DVP camera interface; attaching an OV2640 module will enable end-to-end experiments with real captured images, validating that the stride-based scheme disrupts natural (non-synthetic) JPEG images equally effectively. Natural JPEG images have more varied Huffman table layouts than synthetic test images; it is important to confirm that the stride-at-byte-0 property holds across diverse JPEG generators (e.g., libjpeg vs. ESP32-CAM’s JPEG encoder).

ROI-only partial transmission for true airtime reduction. As established in Section V-C, selective encryption alone does not reduce airtime. A future variant that identifies DCT-sensitive macroblocks and *omits* non-ROI fragments from transmission could reduce fragment count to $\approx 24\%$ of the full image (2 of 9 fragments for a 200×150 image with a small ROI), cutting airtime from 14.9 s to ≈ 3.6 s. This requires the JPEG parser infrastructure that Phase 1 deliberately avoids, and introduces partial-image reconstruction at the receiver.

Per-session key derivation. The current pre-shared key model requires manual key provisioning, which limits scalability. A practical deployment should use HKDF-derived per-session keys: $K_s = \text{HKDF}(K_{\text{master}}, \text{img_seq} \parallel \text{node_id})$. This eliminates the `img_seq` replay risk entirely (F3 gap) without requiring per-session out-of-band key exchange.

Multi-node LoRa mesh evaluation. The E19 and E20 experiments use point-to-point LoRa links. A LoRa mesh deployment with 5–10 nodes introduces routing-layer fragmentation (LoRaMesher distance-vector routing) in addition to application-layer JPEG fragmentation. FC-HMAC must be evaluated in this multi-hop context, where a relay node processes and re-transmits fragments and may be compromised. The formal Tamarin model currently assumes direct sender-to-receiver communication; extending it to a relay model is planned for future work.

VIII. CONCLUSION

We presented Fragment-Chained HMAC (FC-HMAC), a lightweight authenticated selective encryption scheme for JPEG image transmission over LoRa mesh networks. FC-HMAC chains 8-byte HMAC-SHA256 tags across fragments and combines them with a stride-based selective encryption that encrypts 24.2% of image bytes.

The scheme achieves four properties simultaneously:

- 1) **Visual obfuscation:** JPEG decode fails under both naive and format-aware attacker models (E19-PSNR evaluation, format-level disruption confirmed), while the CPU cost is 76% lower than full encryption (2,807 μs vs. $\approx 11,600$ μs for 2,046 bytes on ESP32-S3).
- 2) **Airtime neutrality:** all three encryption modes (ENC_NONE, ENC_DC_ONLY, ENC_FULL) transmit 9 identical-length fragments at 14.9 s total airtime; the efficiency gain is strictly in CPU, not duty cycle.
- 3) **Formal security:** four FC-HMAC lemmas proved in 1.53 s by Tamarin Prover under a Dolev-Yao adversary model, including chain-gap detectability (F2) and no-replay (F3) — properties that per-packet independent HMAC cannot provide.
- 4) **Hardware validation:** 72/72 fragments pass chain verification (100% chain-ok, 100% PDR) on EoRa PI hardware in a 120-second experiment at 922.0 MHz SF9/BW125, with 3.3% per-packet HMAC overhead.

FC-HMAC addresses the ordering and cross-session replay vulnerabilities of per-packet independent HMAC at the cost of one 8-byte tag per fragment — a cost well within the 255-byte LoRa payload budget. Future work will close the F3 firmware gap by adding image sequence numbers to the chain root, validate verifiability under 10–20% packet loss in a three-node relay experiment, and extend hardware measurements to live camera captures.

REFERENCES

- [1] ETSI, “Short range devices (SRD) operating in the frequency range 25 MHz to 1 GHz; part 1: Technical characteristics and test methods,” European Telecommunications Standards Institute, Sophia Antipolis, France, ETSI EN 300 220-1 V3.1.1, 2017.
- [2] Semtech Corporation, *SX1261/2 860 MHz to 930 MHz Low Power Long Range Transceiver Datasheet*, Semtech Corporation, 2021, rev. 2.1. <https://www.semtech.com/products/wireless-rf/lorconnect/sx1262>.
- [3] M. Ryzek and M. Sobieraj, “Secure image transmission system using ESP32-CAM and LoRa module,” *Electronics*, vol. 14, no. 3, p. 512, 2025.
- [4] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The TAMARIN prover for the symbolic analysis of security protocols,” in *Computer Aided Verification (CAV 2013)*, ser. Lecture Notes in Computer Science, vol. 8044. Springer, 2013, pp. 696–701.
- [5] LoRaMesher Contributors, “LoRaMesher: Distance-vector routing for LoRa mesh networks,” <https://github.com/LoRaMesher/LoRaMesher>, 2023, accessed: May 2026.
- [6] ISO/IEC, “Information technology — digital compression and coding of continuous-tone still images — requirements and guidelines,” International Organization for Standardization, Geneva, Switzerland, ISO/IEC Standard 10918-1, 1994.
- [7] M. Bellare, R. Canetti, and H. Krawczyk, “Keying hash functions for message authentication,” in *Advances in Cryptology — CRYPTO 1996*, ser. Lecture Notes in Computer Science, vol. 1109. Springer, 1996, pp. 1–15.
- [8] Espressif Systems, *ESP32-S3 Series Datasheet*, Espressif Systems, 2023, version 1.4. https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf.
- [9] D. Whiting, R. Housley, and N. Ferguson, “Counter with CBC-MAC (CCM),” IETF, RFC 3610, 2003.
- [10] C. Karlof, N. Sastry, and D. Wagner, “TinySec: A link layer security architecture for wireless sensor networks,” in *Proceedings of the 2nd International Conference on Embedded Networked Sensor Systems (SenSys 2004)*. ACM, 2004, pp. 162–175.